

The cassa-photometry Pipeline

From raw pixels to calibrated catalogs, with an error budget the whole way

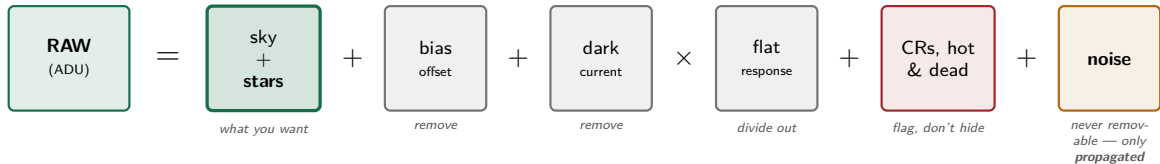
TDMMA Workshop 2026

September 9, 2026

Where we are going

- 1 Motivation & Overview
- 2 The Data Model: SCI / ERR / DQ
- 3 Phase 1: Calibration (ISR)
- 4 Phase 2: Integration & Astrometry
- 5 Phase 3: Photometry & Zero Point
- 6 Phase 4: Diagnostics
- 7 Recap & Hands-on Preview

What is actually in a raw frame?

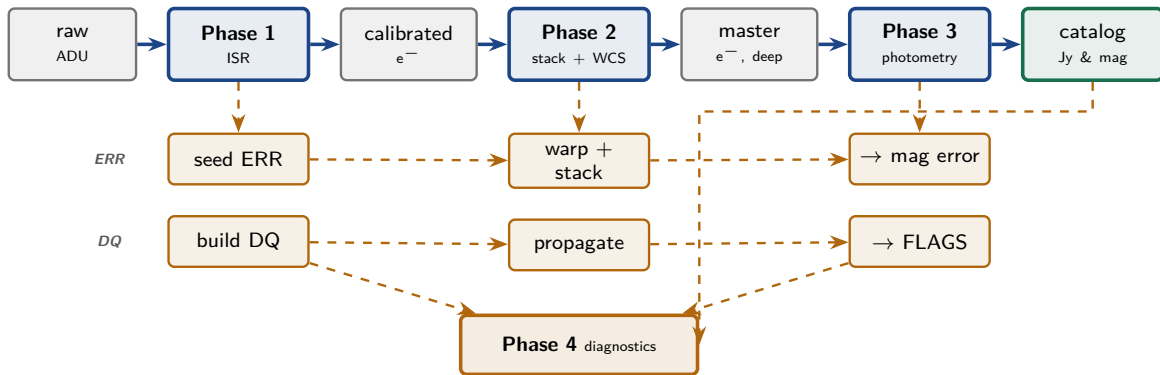


The pipeline's job is to undo the middle three terms, label the fifth, and *track* the last one all the way to the catalog.

The golden rule

A measurement without an error bar is not a measurement.

The whole journey on one slide



The **SCI** spine runs along the top. The **ERR** and **DQ** tracks are not an afterthought — they are created in Phase 1 and updated by every later operation. Phase 4 reads all three back.

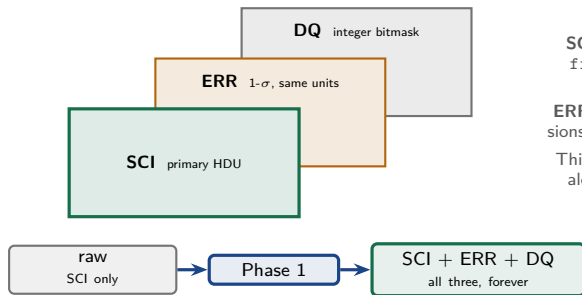
One command per phase

```
cassa-calibrate -i raw/ -o work/phase1      # Phase 1: ISR
cassa-integrate work/phase1 --yes           # Phase 2: align, stack, WCS
cassa-photometry work/phase2               # Phase 3: zero point + catalog
cassa-diagnose  work/phase2 --raw raw/      # Phase 4: diagnostics

cassa-run -i raw/ -o work/                  # Phases 1->2->3 chained
cassa-verify work/phase3                   # independent sanity check
cassa-doctor                               # is this machine ready? (run it first)
```

Each phase is one sub-package writing to its own directory (work/phase1 → phase2 → phase3 → phase4), sharing one config system, logger and FITS model.

Three planes, not one



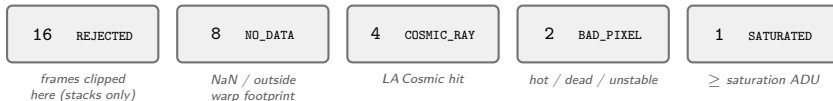
SCI in the *primary* HDU, so DS9, `fits.getdata` and `solve-field` still work on it unchanged.

ERR and DQ travel as named extensions and are rewritten by every stage.

This is the only reason the final catalog can quote a real `Mag_Error`.

once created, the planes are never dropped — only transformed

The DQ bitmask: one integer, many reasons



A pixel that is saturated *and* a cosmic ray:

1 | 4 = 5

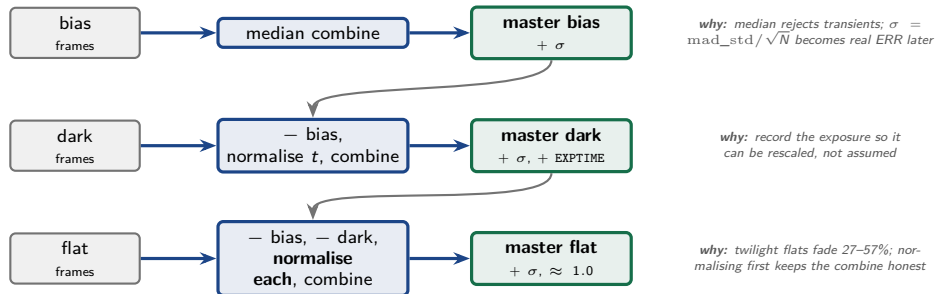
0101

*each catalog source inherits the
OR of every flag inside its footprint*

Not every combination is reachable

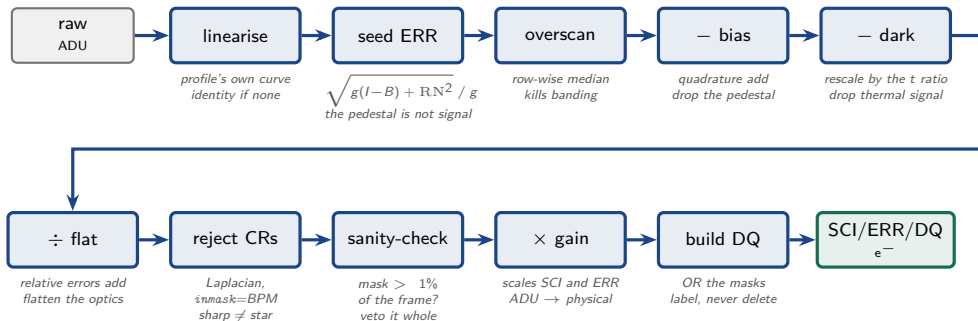
The bad-pixel mask *is* astroscrappy's inmask, so BAD_PIXEL and COSMIC_RAY are disjoint **by construction** — the pipeline never asks whether a known-bad pixel is also a cosmic ray.

Phase 1 — building the masters



Every master is built *with* its own uncertainty — which is precisely what lets ccdproc fold calibration noise into each science frame automatically.

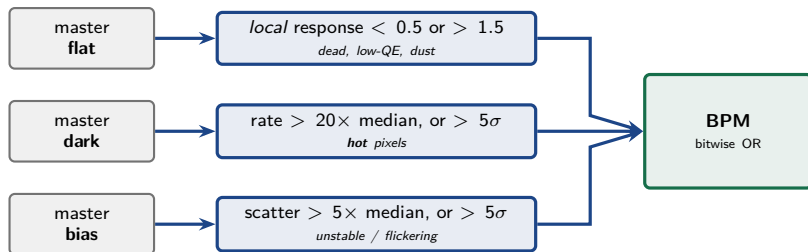
Phase 1 — what happens to one science frame



Library: `ccdproc` does the arithmetic *and* propagates σ at every step.

Cosmic rays arrive at a rate set by the sky, not the field — well under 0.1% of pixels even in a long exposure. A mask claiming more than 1% of the frame is therefore not cosmic rays but the fine-structure model meeting a crowded field, so it is discarded whole and CRVETO written to the header.

Phase 1 — one mask, three witnesses



Why all three? A hot pixel adds $\sim 15 e^-$ to a flat sitting at tens of thousands of ADU. On our data every hot pixel has a normalised flat value of 0.93–1.04 — dead centre. *No flat threshold will ever find it.*

Local, not global: vignetting is optics, not a defect.

Why it matters: the BPM is LA Cosmic's `inmask`. Miss a hot pixel and it is re-detected as a “cosmic ray” in every frame — which is physically impossible, and therefore a free self-test. Flat-only: 197 px flagged CR in ≥ 2 frames, 24 in

all 14. Adding darks: 108 and 5; genuine one-off hits untouched (388 $\rightarrow$ 341).

Phase 1 — seeding the error budget

[math aside — optional]

Poisson (photon shot noise) plus read noise, on the counts the detector *collected* — raw minus the bias pedestal B :

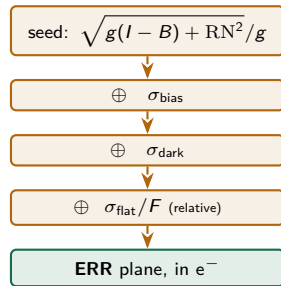
$$\sigma_{\text{ADU}} = \frac{\sqrt{g \cdot (I - B) + \text{RN}^2}}{g}$$

Treating the pedestal as signal would inject tens of fictitious electrons into *every* pixel. Subtracting a master adds in quadrature:

$$\sigma_{\text{out}} = \sqrt{\sigma_{\text{sci}}^2 + \sigma_{\text{bias}}^2}$$

Dividing by the flat propagates *relative* errors:

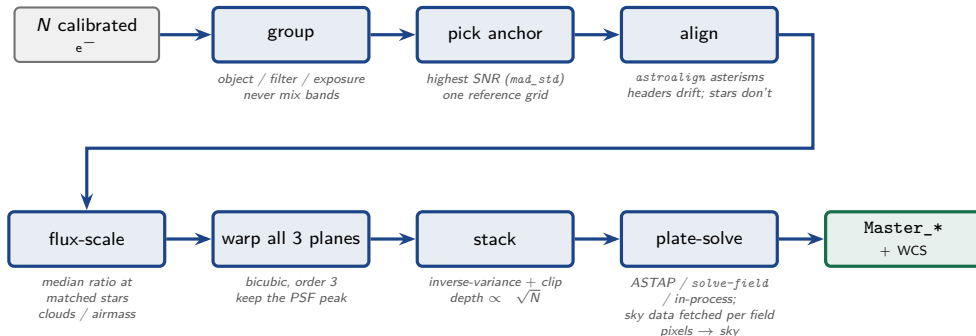
$$\left(\frac{\sigma_{\text{out}}}{S_{\text{out}}} \right)^2 = \left(\frac{\sigma_{\text{sci}}}{S} \right)^2 + \left(\frac{\sigma_{\text{flat}}}{F} \right)^2$$



Because the flat term is *relative*, an over-estimated master flat inflates the error bars on

bright stars hardest — exactly where photometry is done.

Phase 2 — what happens to a set of frames

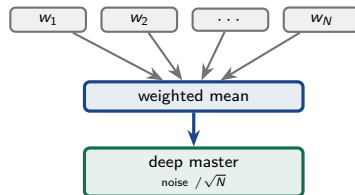


The ERR plane is warped and stacked *with* the data; DQ rides along and gains NO_DATA wherever the footprints stop overlapping.

Weight frame i by $w_i = 1/\text{var}_i$:

$$\text{master} = \frac{\sum_i w_i \text{sci}_i}{\sum_i w_i}, \quad \text{var}_{\text{master}} = \frac{1}{\sum_i w_i}.$$

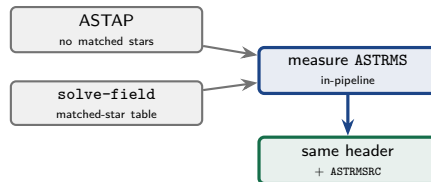
- Noisy frames contribute less; this is the *optimal* linear combination.
- N equal frames $\Rightarrow$ noise shrinks by $\sqrt{N}$.
- Rejection: sigma-clip at ≥ 5 frames, min/max below.



Phase 2 — the plate solve: three backends, one answer

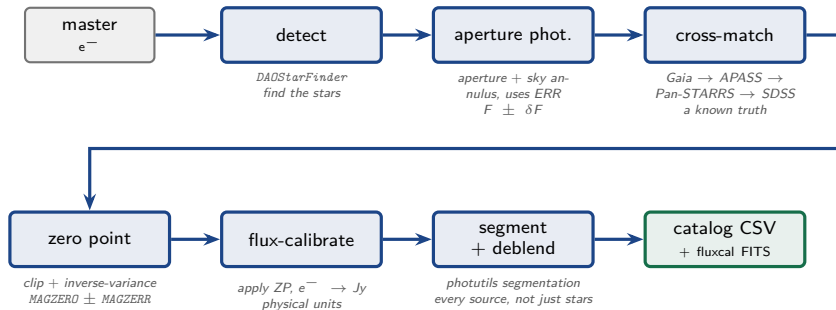
Backend	Where it runs
ASTAP	everywhere, ARM included
solve-field	Linux x86-64, Intel macOS
in-process	no ARM Linux wheel

- The installer tries them *in that order*, the first that installs wins.
- Sky data is fetched **per field**: ~6 MB (ASTAP) or ~246 MB (Astrometry.net), out of 859 MB / 34 GB published.
- Repointing costs only what the new field adds.



Phase 3 sizes its cross-match from ASTRMS, so the pipeline *measures* it rather than trusting the solver — otherwise your photometry would depend on which solver you installed.

Phase 3 — from counts to calibrated magnitudes



Output columns: MAG_BEST, MAG_PSF, MAG_AUTO, MAG_APER, MAG_ISO, CLASS, SNR, FLAGS — the DQ bits OR-ed inside each source footprint. With no zero point every one of those is NaN and MAG_INST carries the measurement instead: a failed cross-match costs you the scale, not the photometry.

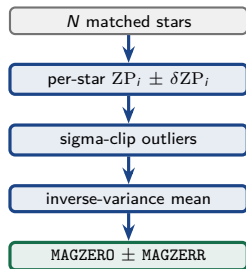
For each catalog-matched star:

$$m_{\text{inst}} = -2.5 \log_{10}(F), \quad \delta m_{\text{inst}} \approx 1.0857 \frac{\delta F}{F}$$

$$\text{ZP}_i = m_{\text{cat}} - m_{\text{inst}}, \quad \delta \text{ZP}_i = \sqrt{\delta m_{\text{inst}}^2 + \delta m_{\text{cat}}^2}$$

Combine with sigma-clipping and $w_i = 1/\delta \text{ZP}_i^2$; report MAGZERR as the *larger* of the formal error and the scatter's standard error. Final source magnitude:

$$\delta m = \sqrt{(1.0857 \delta F / F)^2 + \text{MAGZERR}^2}$$



Which magnitude?

[math aside — optional]

An aperture never catches *all* of a star's light, and how much it misses depends on the source.

MAG_ISO is **isophotal** — the flux above the detection threshold. A bright star's footprint reaches further down its wings than a faint one's, so the fraction captured *depends on brightness*.

Applying an aperture-derived zero point to it therefore **tilts** the magnitude scale. A tilt is not an offset: no single correction removes it.

Measured against simulated data with known truth.

column	median	trend
MAG_BEST	+0.03	+0.004
MAG_APER	−0.01	−0.011
MAG_ISO	+0.53	+ 0.348

trend in mag per mag

Which column, then?

MAG_BEST — PSF for point sources, Kron for extended — is the *total* magnitude. MAG_APER is measured in the same aperture the zero point was, so it is the one exactly consistent with the calibration. MAG_ISO is kept only for comparison with older catalogs.

The discriminant is $\text{MAG_PSF} - \text{MAG_AUTO}$: how much of a source's light lies outside the PSF core.

Not ellipticity. That is *shape*: a face-on elliptical galaxy is round and passes as a star; a slightly trailed star is elongated and fails.

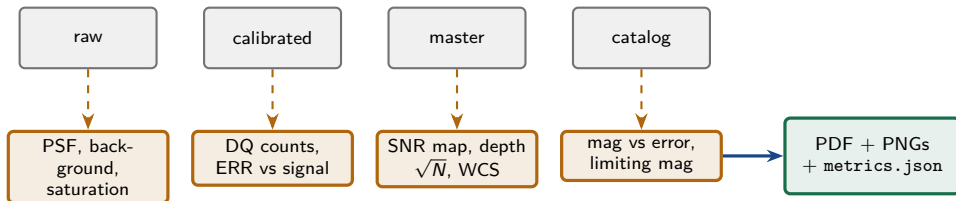
The threshold is calibrated **per frame** from the stellar locus itself — so it adapts to tonight's seeing, with no training data.

AMBIGUOUS is an answer

Below the SNR where the locus separation falls under its own scatter, no class is asserted. CLASSLIM records where that happens, rather than pretending to classify to the detection limit.

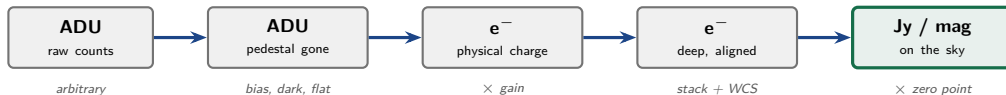
Why it matters beyond the catalog column: a galaxy leaking into the zero point biases every magnitude in the frame.

Phase 4 — did it actually work?



Phase 4 does not change a single pixel — it *reads back* the error budget you have been carrying since Phase 1 and asks whether it behaves. The ERR-vs-signal panel should trace $\sqrt{S}$; the depth boost should approach $\sqrt{N}$. When they do not, something upstream is wrong.

The units journey



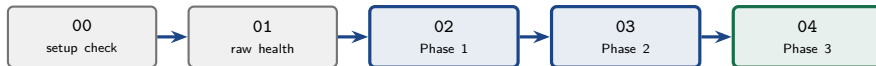
What never changes

Three planes travel together the entire way. Every operation that touches SCI also touches ERR and DQ.

What to remember

Phase 1 *seeds* the error, Phase 2 *stacks* it optimally, Phase 3 *converts* it to magnitudes, Phase 4 *checks* it.

In the hands-on session



- Notebooks 02–04 each end with **two exercises**, part-written: you fill in the lines marked `TODO`, and a worked solution surrounds them.
- Notebook 01 reads only the raw frames and returns a go/no-go verdict; 02–04 each consume the previous notebook's output.
- Phase 2 plate-solves against a star catalog, fetched **per field** and then cached — whichever of the three backends you installed; Phase 3's cross-match is a **live** catalog query.

Questions?

Next: the hands-on notebooks.